

Name: Ambika Kashyap

UID : 23BCC70026

Class : 23BCC-1(A)

Experiment – 6.3

Aim

To implement **Role-Based Access Control (RBAC)** in a Node.js application using **JWT authentication**, ensuring that different users (Admin, User, Moderator) have access only to authorized resources.

Theory

◇ Role-Based Access Control (RBAC)

RBAC is a method of restricting system access based on user **roles**.

Each role defines a set of permissions determining what actions a user can perform.

Example:

- **Admin:** Can manage users and view all data.
- **Moderator:** Can edit or delete content but cannot manage users.
- **User:** Can only view their own content.

◇ Key Concepts

1. **Authentication:** Verifies the user's identity (via JWT token).

2. **Authorization:** Determines what actions that user is allowed to perform.
3. **JWT (JSON Web Token):** Used to securely transmit user identity and role information between client and server.
4. **Middleware:** Used to check both authentication (token validity) and authorization (role permission).

◇ How RBAC Works

1. User logs in and receives a **JWT** containing their **role**.
2. Each route defines which roles can access it.
3. Middleware verifies both:
 - a. Token validity (user is authenticated)
 - b. Role permission (user is authorized)
4. Unauthorized users are denied access.

Procedure

1. Set up an Express server and install dependencies (express, jsonwebtoken, dotenv, body-parser).
2. Create a /login route that issues a JWT token containing the user's role.
3. Implement middleware:
 - a. verifyToken → Validates JWT.
 - b. authorizeRoles → Checks if the user has required permissions.
4. Create routes restricted by roles (Admin, User, Moderator).
5. Test each route using valid and invalid tokens.

Code Implementation

// File: server.js

```
const express = require("express");  
const jwt = require("jsonwebtoken");
```

```

const bodyParser = require("body-parser");
require("dotenv").config();

const app = express();
app.use(bodyParser.json());

const SECRET_KEY = process.env.JWT_SECRET || "mysecretkey";

// ✅ LOGIN ROUTE - Generates token with role
app.post("/login", (req, res) => {
  const { username, role } = req.body;

  // roles: "admin", "user", "moderator"
  if (!username || !role) {
    return res.status(400).json({ message: "Username and role are required" });
  }

  const token = jwt.sign({ username, role }, SECRET_KEY, { expiresIn: "1h" });
  res.json({ message: "Login successful", token });
});

// ✅ Middleware: Verify JWT Token
function verifyToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];
  if (!token) return res.status(403).json({ message: "Token missing" });

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) return res.status(401).json({ message: "Invalid token" });
    req.user = user;
    next();
  });
}

// ✅ Middleware: Check Role Authorization
function authorizeRoles(...allowedRoles) {
  return (req, res, next) => {

```

```
    if (!allowedRoles.includes(req.user.role)) {  
      return res.status(403).json({ message: "Access denied: Insufficient role" });  
    }  
    next();  
  };  
}
```

//  Routes with Role-Based Protection

// Public route

```
app.get("/", (req, res) => {  
  res.send("Welcome! Please login to access protected routes.");  
});
```

// Admin-only route

```
app.get("/admin", verifyToken, authorizeRoles("admin"), (req, res) => {  
  res.json({  
    message: `Hello ${req.user.username} (Admin)`,  
    data: "You can manage all users and system settings.",  
  });  
});
```

// Moderator route

```
app.get("/moderate", verifyToken, authorizeRoles("admin", "moderator"), (req, res) => {  
  res.json({  
    message: `Hello ${req.user.username} (Moderator)`,  
    data: "You can edit or delete posts.",  
  });  
});
```

// User route

```
app.get("/profile", verifyToken, authorizeRoles("admin", "moderator", "user"), (req, res) => {  
  res.json({  
    message: `Hello ${req.user.username} (${req.user.role})`,  
    data: "You can view your profile and posts.",  
  });  
});
```

```
const PORT = 5000;  
app.listen(PORT, () => console.log(` Server running on port ${PORT}` ));
```

Sample Outputs

Login Request

POST → /login

Body:

```
{  
  "username": "Nikhil",  
  "role": "moderator"  
}
```

Response:

```
{  
  "message": "Login successful",  
  "token": "eyJhbGciOiJIUzI1NiIsInR..."  
}
```

Access as Moderator

GET → /moderate

Header:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...

Response:

```
{  
  "message": "Hello Nikhil (Moderator)",  
}
```

```
"data": "You can edit or delete posts."
}
```

Unauthorized Access

GET → /admin using a User token

Response:

```
{
  "message": "Access denied: Insufficient role"
}
```

Learning Outcomes

After completing this experiment, students will be able to:

1. Understand **role-based access control (RBAC)** and its importance in secure systems.
2. Implement **JWT authentication** and **authorization** in Node.js.
3. Differentiate between **authentication** (who the user is) and **authorization** (what they can do).
4. Use **Express middleware** effectively for verifying roles and permissions.
5. Develop scalable and secure APIs that follow **principle of least privilege**.